MIRAMAR LABS

HYBRID ON-PREM / GCP AI PLATFORM

Alpaca Options Trading Agents

A three-agent trading floor that turns a directional call into a specific options contract via an Alpaca MCP tool-calling agent — every LLM decision running as local inference on one NVIDIA DGX Spark — a machine running on 100% solar power.

Alpaca AI Trading Agents Hackathon · lablab.ai · Sep 2026 · paper account PA3MAA2HJLUG · $100,000

POWERED BY
NVIDIA DGX Spark
ON 100% SOLAR POWER

THE PROBLEM

A directional signal is not a trade

"AAPL looks like a buy" is the easy half. Turning it into an options position is a second, harder decision — and normally a human options trader's job.

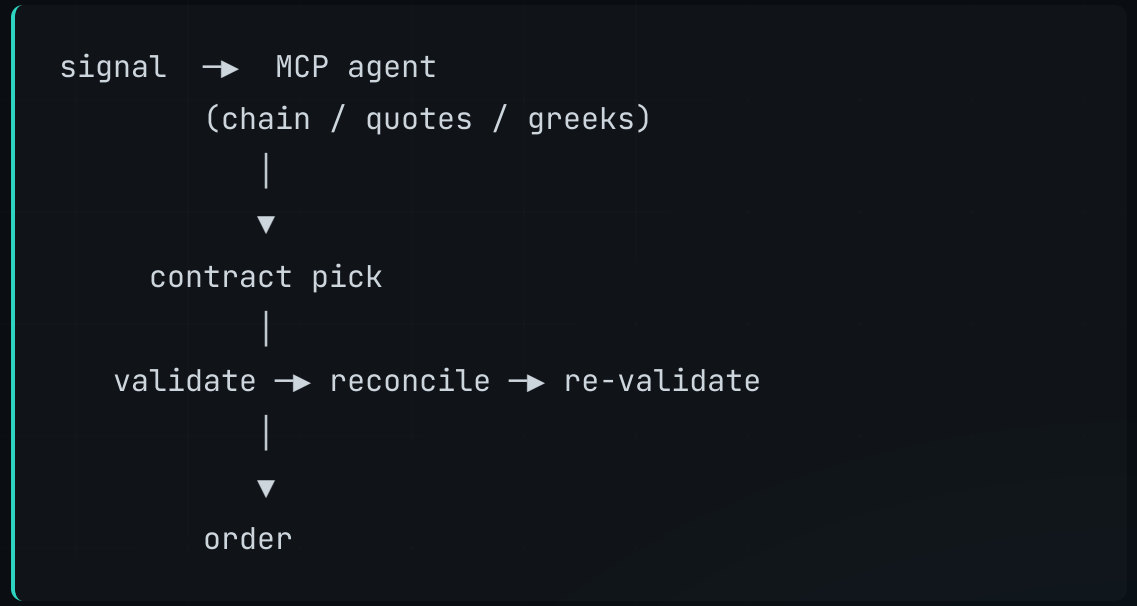
- **Which expiration?** Too short and theta eats you; too long and you overpay for time you don't need.
- **Which strike?** The delta has to match the conviction — a 0.30 lotto ticket and a 0.55 directional play are different trades.
- **Is the premium sane?** Wide spreads, thin open interest, and IV crush around events turn a good call into a losing fill.
- **Does it fit the book?** Notional exposure, existing positions, macro-event timing — all before a single order.

THE IDEA

Put an LLM agent in the contract-selection seat

Give it live read access to Alpaca's option chain, quotes and Greeks through MCP and let it reason to a specific contract — with a deterministic layer underneath that assumes the model can be **wrong or adversarial**.

- **The agent** browses the chain and proposes one contract: strike, expiration, delta, premium, and its reasoning.
- **The code** re-derives every hard constraint from the chain data itself — the model's numbers are never trusted, only its rationale.
- **A structured-output miss** degrades to a deterministic picker, never to “no trade.”



ARCHITECTURE

Three agents, one continuous floor

CRONJOB · 08:55 ET

Analyst

LangGraph pipeline: screens movers + large caps, pulls news, indicators and its own track record, emits the day's watchlist with a per-symbol budget.

→

POLL LOOP · EVERY 600S

Dealer

Per symbol: BUY / HOLD / SELL + confidence. Every non-HOLD goes to the MCP contract-selection agent.

ALPACA MCP

→

FASTAPI EXECUTOR

Floor Broker

The only writer to the Trading API. Re-validates server-side, places the order, runs fill / synthetic-exit / restart-recovery loops.

- State passes through a **portfolio ConfigMap** (Analyst → Dealer) and **Postgres** — the agents' memory across cycles: the Analyst and Dealer read their own past picks, decisions and execution outcomes back into their prompts. Also the audit trail and order reconciliation.
- Runs live on `k3s`, namespace `alpaca-options-trader` — not a backtest, not a one-off demo script.

THE CENTREPIECE

The Alpaca MCP integration

- `mcp_options.py` launches `alpaca-mcp-server` over stdio via `langchain-mcp-adapters`.
- **Read-only toolset** — `assets`, `options-data`, `account`. The Dealer can inspect the chain but **can never place an order**.
- Inputs: the underlying, the direction (call = bullish, put = bearish; always long), and the config DTE / delta windows.
- The LLM drives a **bounded ≤6-round** tool-calling loop over live chain / quotes / Greeks.
- Returns a structured pick: strike, expiration, delta, mid-price premium, reasoning.

```
round 1  get_option_chain(AAPL)
round 2  filter DTE 15-45
round 3  get_quotes(candidates)
round 4  get_greeks(shortlist)
round 5  compare delta / spread
round 6  → OptionContractPick
         { strike, exp, delta,
           premium, reasoning }
```

TRUST BOUNDARY

Validate → reconcile → re-validate

Everything downstream of the model call assumes the model is adversarial, not just wrong.

1. **Validate** — reject a pick that claims the wrong option type for the signal, or names a contract that was never seen in a chain response (a hallucination).
2. **Reconcile** — overwrite strike / expiration / delta / premium with what the chain actually showed for that exact contract. Only the reasoning text is trusted from the model.
3. **Re-validate** — server-side in Floor Broker, against the live DTE / delta windows and a fresh ask quote, immediately before the order is submitted.

This also blocks the interesting failure modes for free: a real contract with a **fabricated premium**, or a call-labelled pick whose OCC symbol is actually a put.

RISK MANAGEMENT

Eight gates between "BUY" and an order

Each one deterministic — none of them is an LLM call.

1. Confidence threshold on the signal
2. Macro-event blackout calendar (FOMC / CPI / jobs / PCE)
3. Same-symbol stop-loss cooldown
4. Portfolio win-rate throttle
5. Daily profit-target / loss-limit halt
6. Operator kill switch — flipped in config, no redeploy
7. DTE / delta window re-validation
8. Hard notional cap, priced against the live market

Once a position is open, Alpaca has no server-side bracket for options — so Floor Broker runs a **30-second poll loop** that force-closes on a **50% loss**, a **175% gain**, or expiration reaching **3 DTE**, whichever comes first, regardless of whether new entries are currently gated.

MIRAMAR LABS

INFRASTRUCTURE

Everything on one NVIDIA DGX Spark

k3s cluster

All four workloads in namespace `alpaca-options-trader` — Analyst, Dealer, Floor Broker, EOD report.

Ollama `qwen2.5:32b-instruct`

Local inference behind every Analyst / Dealer / MCP decision — on a DGX that runs on 100% solar power. No external LLM API.

Postgres

Shared platform instance — the agents' memory across cycles (past picks, decisions and outcomes feed the next run), plus audit trail and reconciliation.

GitHub Actions runner

Self-hosted. push → test + lint → build 4 images → rolling deploy.

`config.yaml` is fetched fresh from GitHub every 60 seconds by every pod — risk thresholds, blackout dates, even the LLM model are a plain `git push`, no rebuild.

MIRAMAR LABS

RESULTS — 4 SEP 2026 CLOSE, FINAL DAY

Live, risk-gated, and still standing

$101,299
EQUITY AT CLOSE
+1.3% VS $100k OPEN

+$2,270
REALISED, 2/2 EXITS
BOTH WINNERS

3 / 5
OPEN POSITIONS GREEN
BEST AMZN 255C +15%

3 / 4
SESSIONS CAPPED BY A GATE
PROFIT-TARGET · BLACKOUT · EXITS

- Trading **live and continuously since 1 September** — not a backtest. All seven option positions were opened in one session on day 1, every one **filled within 30s** of the signal.
- Every session since has been capped by a **different risk gate**, not a lack of signal: the daily profit-target halt rejected 16 real BUY signals on day 2; the Floor Broker's synthetic take-profit loop closed its first two positions for **+$2,270 realised** on day 3 (Alpaca has no server-side option brackets — that loop is ours); a scheduled macro-event blackout (NFP) blocked all new entries on day 4.
- 3 of 5 open positions are green; the laggard, V 380C at −35.4%, is still short of the −50% synthetic stop. Live P/L + model badges in the README.

Structured output is the hard part — not the MCP plumbing

- The tool-calling loop works every cycle: the agent calls the chain / quote / Greeks tools and pulls real data.
- But the closing `.with_structured_output()` returned **empty every cycle** on the first model, `qwen3.6:35b-a3b` — a 35B MoE with only ~3B active. Root cause was that active-parameter count, not context: the failing prompt was ~4k tokens against a 32k window.
- All 7 day-1 contracts came from the fallback path. **Fixed inside the window:** each candidate run through the real selection path against live MCP data (committed as `scripts/check_contract_selection.py`) — switched to dense `qwen2.5:32b-instruct-q4`, which returns valid structured picks (`fell_back=False` on NVDA / TSLA / AAPL). Selection now runs on the model.

_fallback_pick stays a first-class safety net: it selects from the exact chain rows the loop already fetched, under the same gates — any future empty result degrades to "the code skims the chain like a human would," never to nothing. The bulletproof long-term answer is a guided-decoding backend (vLLM / SGLang / NIM) — an infra migration off Ollama that didn't fit the window.

MIRAMAR LABS

THANK YOU

An LLM in the loop, a deterministic layer under it

REPO github.com/miramar-labs/alpaca-options-trading-agents

LICENSE MIT, public — real, dated commits across the competition week

ACCOUNT Dedicated Alpaca paper account PA3MAA2HJLUG — $100,000, trading continuously since 1 Sep

STACK Alpaca · Model Context Protocol · alpaca-mcp-server · LangGraph · Ollama · FastAPI · PostgreSQL · k3s

POWERED BY
NVIDIA DGX Spark
ON 100% SOLAR POWER